

KIẾN TRÚC ĐỊNH THỜI LOGIC PHÂN TÁN (STRICT VS HEURISTIC)

Tài liệu Tóm tắt Thiết kế — Bài toán #112 Log Delay Compensator (2-Page Executive Brief)

1 Kiến trúc Tổng thể & Bài toán

Hệ thống xử lý luồng log Web Server đến *out-of-order* 24/7, dùng **Watermark** để quyết định chờ dữ liệu trễ bao lâu trước khi chốt cửa sổ (Tumbling Window 5s trên Event-Time). Deliverable cốt lõi là đường cong đánh đổi **Data Completeness % vs Wait Time**. Hệ thống được phân rã thành các phân lớp độc lập, dùng chung một đường dữ liệu (data plane) nhưng tách hai mặt phẳng điều khiển (control plane) cho hai chiến lược:

- Đặc tả Dữ liệu và Cửa sổ:** Mỗi bản ghi log Web Server chứa mốc thời gian sự kiện logic T_{event} và mốc thời gian tiếp nhận vật lý $T_{arrival}$. Độ trễ truyền dẫn được xác định là $\ell = T_{arrival} - T_{event}$. Cửa sổ Tumbling Window được định nghĩa là $[W_{start}, W_{end})$ với kích thước cố định $W_{end} - W_{start} = 5$ giây, bắt đầu tại các mốc thời gian chia hết cho 5 ($W_{start} = 5k, k \in \mathbb{Z}$).
- Ingest (Tạo dữ liệu & Token):** Ingestor gán mốc sự kiện T_{event} , tính phân mảnh ngang PartitionID = $\text{hash}(\text{host}) \bmod 12$, phát vào dòng sự kiện Kafka. Ở chế độ Strict, Ingestor chèn các Punctuation Token mang mốc cam kết T_{commit} (hoặc Empty Punctuation Token khi phân vùng rảnh) để đánh dấu mốc thời gian an toàn.
- Data Plane (Kafka Broker):** Gồm 12 phân vùng độc lập, RF=3, lưu trữ dạng append-only. Đóng vai trò là log phân tán bền vững, bảo toàn thứ tự ghi nhận và cho phép replay từ offset bất kỳ sau sự cố.
- Compute (Tầng xử lý - 4 Worker):** Phân bổ 3 phân vùng/Worker. Mỗi phân vùng được xử lý bởi một engine độc lập. Worker sử dụng cơ chế Backpressure để tạm dừng nạp log khi hàng đợi nội bộ quá tải, tránh tràn bộ nhớ.
- Control Plane (Mặt phẳng điều khiển):** Strict dùng Coordinator HA (Raft / ZooKeeper) để tính toán watermark toàn cục W_{global} đồng bộ, bảo đảm tính nhất quán mạnh. Heuristic dùng Aggregator HA (Active-Standby qua khóa ZooKeeper) để thu thập watermark cục bộ W_{heur} bất đồng bộ, tối ưu hóa độ trễ.
- Tiered Storage (Lưu trữ phân tầng):** Tầng Tier-1 (RocksDB Local) phục vụ hot path; Tier-2 (Shared Checkpoint Volume) lưu snapshot trạng thái và offset Kafka phục vụ failover; Tier-3 (MinIO/S3 Cold Archive) lưu trữ lịch sử lâu dài.

2 Thiết kế Tuyển Strict Watermark (0% Loss)

Strict Path phục vụ nghiệp vụ kiểm toán/đối soát tài chính, ưu tiên tính đúng đắn tuyệt đối; mốc thời gian toàn cục chỉ tiến khi *mọi* phân vùng đang hoạt động đã vượt qua mốc đó. Theo Özsu & Valduriez, đây là thiết kế **PC/EC** (chọn Consistency cả khi có Partition lẫn khi Else), chấp nhận chi phí điều phối cao và độ trễ tăng để bảo đảm tính nhất quán mạnh.

- Định thời dựa trên Punctuation:** Ingestor chèn token kiểm soát mang mốc cam kết T_{commit} . Khi phân vùng rảnh (Idle), Ingestor vẫn gửi *Empty Punctuation Token* mỗi 1000ms để dòng thời gian logic toàn cụm không bị đóng băng — đây là cơ chế *chính* giữ W_{global} luôn tịnh tiến, giải quyết bài toán *không thể phân biệt phân vùng nhàn rỗi với phân vùng tắc nghẽn* (một vấn đề cốt lõi trong hệ phân tán theo Özsu & Valduriez).
- Worker-side Bounded Priority Queue (Sắp xếp Heap):** Sắp xếp cục bộ theo heap nhị phân trong bộ nhớ với kích thước tối đa $K = 10.000$ sự kiện, sử dụng bộ đếm tăng dần làm tiêu chí phụ phá hòa khi trùng mốc sự kiện. Công thức điều kiện Forced Pop: khi heap đạt kích thước tối đa ($size \geq K$) hoặc khoảng cách thời gian xử lý giữa đỉnh heap và event mới vượt quá ngưỡng cho phép ($t_{\text{current_event}} - t_{\text{heap_top}} > 1000\text{ms}$), sự kiện cũ nhất sẽ bị đẩy ra để giải phóng RAM, bảo đảm độ trễ xử lý được giới hạn.
- Hội tụ toàn cục tối thiểu (Min-of-Min):**

$$W_{global}(t) = \max \left(W_{global}(t-1), \min_{\forall P_k \in \text{Active}} LW_i(P_k) \right) \quad (1)$$

Phân vùng phản hồi chậm (STALE) và nhàn rỗi ngầm (IDLE) *vẫn* tham gia $\min()$ để giữ cam kết 0% loss. Chỉ phân vùng xác nhận lỗi (FAILED) hoặc được Worker đánh dấu nhàn rỗi tường minh (explicit idle) sau ngưỡng 30s mới bị loại trừ; khi có sự kiện mới, phân vùng tự động quay lại hàm tính — đây là cơ chế **Idleness Bypass** chống Straggler.

- Chống Split-Brain (Fencing):** Mọi lệnh điều phối kèm cặp ($term, command_id$) tăng đơn điệu. Worker duy trì nhiệm kỳ lớn nhất đã ghi nhận $term_{max}$. Mọi lệnh điều phối nhận được có $term < term_{max}$ sẽ bị từ chối thẳng thừng để ngăn chặn lệnh reassign lạc hậu từ Coordinator cũ khi bị phân tách mạng, và bỏ qua lệnh trùng $command_id$ (idempotent).
- Tái phân bổ cân bằng tải:** Khi nhịp tim quá hạn (ngưỡng 10s), Coordinator phát hiện Worker chết, tăng nhiệm kỳ ($term$) của Fencing Token, chia *đều* các phân vùng mỡ côi cho các Worker sống (ví dụ P9→node0, P10→node1, P11→node2) — tránh dồn tải gây sập dây chuyền. Khi ≥ 2 Worker sập, Cascading Failure Protocol điều phối qua nhật ký đồng thuận để tái phân bổ toàn bộ phân vùng mỡ côi.
- Strict Failback State Machine:** Bàn giao ngược 5 bước trên nhật ký đồng thuận: PAUSE → FLUSH-ACK → REASSIGN → SEEK-RESUME → COMPLETE, qua chu trình trạng thái ASSIGNED → REASSIGNING → PAUSED → ASSIGNED. Bảo đảm tại một thời điểm duy nhất một Worker được tiêu thụ/ghi một phân vùng (Exactly-Once output). Kết quả được phát song song vào dòng kết quả chính và dòng kiểm toán (Critical Audit Sink) để bộ tiêu thụ kiểm toán replay độc lập.
- Exactly-Once input:** Hai vòng lọc khi replay — Watermark Filter ($T_{event} < W_{global} \Rightarrow$ loại bỏ) và bảng băm khử trùng lặp theo định danh sự kiện (có TTL) trong RocksDB; Worker tiếp quản tiếp tục từ offset kế tiếp dựa trên checkpoint Tier-2.

3 Thiết kế Tuyển Heuristic Watermark (Low Latency)

Heuristic Path loại bỏ điều phối đồng bộ chặn, trao quyền tự trị cục bộ tối đa cho Worker (Özsu & Valduriez: **Site Autonomy** giúp giảm phụ thuộc vào điều phối trung tâm, cải thiện hiệu năng và giảm trễ) và bù dữ liệu muộn bất đồng bộ

qua DLQ. Đây là thiết kế **PA/EL**: khi Partition chọn Availability, khi Else chọn Latency, chấp nhận nhất quán sau cùng (Eventual Consistency) để đạt độ trễ cực thấp.

- **Ước lượng phân vị thích ứng DDSketch (Hàm log)**: DDSketch chia độ trễ $\ell = T_{arrival} - T_{event}$ thành các bucket theo hàm logarithmic:

$$I = \left\lfloor \log_{\gamma} \left(\frac{\ell}{\sigma} \right) \right\rfloor \quad (2)$$

với cơ số $\gamma = 1.01$ (sai số tương đối bảo đảm 1%) và σ là hệ số tỷ lệ. Sketch không chế bộ nhớ cực kỳ nhỏ gọn dưới 100KB nhờ giới hạn cứng 1024 bucket. DDSketch có tính *cộng gộp* (mergeable), cho phép Aggregator gộp các sketch từ nhiều phân vùng để ước lượng phân vị toàn cục mà không làm suy giảm độ chính xác của phân phối độ trễ.

- **Watermark cục bộ thích ứng (đơn điệu)**:

$$W_{heur,i}^{P_k}(t) = \max \left(W_{heur,i}^{P_k}(t-1), T_{event_max} - L_{eff}(t) \right), \quad L_{eff} = \text{DDSketch.quantile}(p) \quad (3)$$

Phân vị tự nâng từ $p_{normal} = 0.99$ lên $p_{safe} = 0.999$ khi phát hiện burst lag $> 2.0 \times$ trung bình, mở rộng biên an toàn (Loss Budget Guarantee).

- **Cold Start & Negative Lag**: 3 pha khởi động (buffer-only $\rightarrow$ Conservative Prior $L_{max} = 60s \rightarrow$ Normal khi $\geq 10s$ và ≥ 1000 mẫu); lọc lag âm do lệch đồng hồ (Clock Skew), hiệu chuẩn cộng bù $|\text{median_neg_lag}|$ khi tỷ lệ âm $\geq 1\%$.
- **Aggregator HA (Active-Standby)**: Sử dụng khóa loại trừ trên ZooKeeper hoặc khóa tệp dự phòng (hỗ trợ đa nền tảng). Active ghi nhịp tim mỗi 1s. Nếu nhịp tim mất hoặc quá hạn ($> 1.5s$), Standby tự động chiếm khóa, khôi phục trạng thái đã lưu bền vững và lên Active ($RTO \leq 2s$), tránh SPOF.
- **Replay-Mode Sketch Snapshot & Rollback**: Hàng đợi 6 snapshot DDSketch trong 60s gần nhất. Khi phát hiện trễ tăng $10 \times$ (dấu hiệu replay), Worker rollback về snapshot an toàn và đồng bộ bộ ước lượng, tránh “nhiễm bẩn” thống kê. Kết hợp Replay Sub-Checkpointing để không phải replay lại từ đầu khi xử lý backlog lớn.
- **Đường ống sửa lỗi DLQ Correction**: Bản ghi $T_{event} < W_{global_h}$ (đến muộn) định tuyến vào DLQ. Bộ tiêu thụ DLQ nạp lại trạng thái lịch sử từ Tier-3, tính Delta cho các window bị ảnh hưởng, phát thông điệp sửa lỗi (Correction Message) dạng delta hoặc versioned updates xuống downstream $\Rightarrow$ Eventual Completeness đạt 100% mà không gây tắc nghẽn.

4 Tiered Storage & Cơ chế Failover/Failback

Hệ thống lưu trữ phân tầng (*Tiered Storage*) đóng vai trò làm nền tảng kỹ thuật cho việc chuyển giao trạng thái an toàn, cô lập giữa các Worker. Toàn bộ state và offset được định danh theo **partition_id** thay vì tên tiến trình node (Site Autonomy):

- **Cấu trúc Lưu trữ 3 Tầng (3-Tier Storage)**:
 - *Tier-1 (RocksDB Local)*: Lưu trữ trạng thái tạm tính của các cửa sổ chưa đóng, aggregate nóng (ví dụ sum, count) và bảng băm dedupe (lọc trùng lặp dựa trên định danh sự kiện kèm thời gian sống TTL). Phục vụ hot path đọc/ghi tức thời với độ trễ cực thấp ($< 1ms$).
 - *Tier-2 (Shared Checkpoint Volume)*: Thư mục dùng chung lưu trữ snapshot định kỳ 10s của RocksDB (dưới dạng tập SST) và offset consumer Kafka. Đây là điểm khôi phục nhanh (RTO thấp) giúp Worker gánh hộ nhanh chóng nạp lại trạng thái mà không cần đọc lại toàn bộ log từ Kafka.
 - *Tier-3 (MinIO/S3 Cold Archive)*: Lưu trữ các cửa sổ đã chốt hoàn thiện dưới dạng tệp tin định dạng JSON theo khóa tất định **partition/window_id**. Phục vụ truy vấn lịch sử, đối soát hậu kỳ và khôi phục thảm họa (Disaster Recovery).
- **Vòng đời trạng thái Closed Window (Tiered Eviction)**: Khi watermark cho phép chốt cửa sổ, trạng thái chuyển dịch tuần tự: CLOSED $\rightarrow$ UPLOADING (đang tải lên S3) $\rightarrow$ UPLOADED (S3 xác nhận thành công) $\rightarrow$ PURGED (dọn dẹp khỏi RocksDB local). Sử dụng khóa tất định trên S3 giúp việc ghi có tính idempotent tuyệt đối khi replay.
- **Cơ chế Failover (Khắc phục sự cố nhanh)**: Khi Worker chết (heartbeat timeout quá 10s), Coordinator/Aggregator đánh dấu cô đơn các phân vùng. Node sống được gán gánh hộ sẽ tải trực tiếp checkpoint Tier-2 tương ứng của phân vùng, khôi phục trạng thái RocksDB local, seek consumer Kafka đến **last_safe_offset + 1** và chạy tiếp. Hoàn toàn không phụ thuộc trạng thái trong RAM của node đã chết.
- **Quy trình Failback 5 bước (Bàn giao an toàn)**: Bàn giao ngược phân vùng từ node gánh hộ về node cũ đã hồi phục qua 5 bước nghiêm ngặt do Coordinator điều phối nhằm bảo đảm nguyên tắc Exactly-Once (tại một thời điểm chỉ có duy nhất một Worker tiêu thụ phân vùng):
 1. **PAUSE**: Coordinator gửi lệnh yêu cầu node gánh hộ dừng tiêu thụ dữ liệu từ Kafka.
 2. **FLUSH-ACK**: Node gánh hộ flush toàn bộ state nóng từ Tier-1 ra Tier-2, chốt Kafka offset an toàn cuối cùng, rồi gửi ACK xác nhận.
 3. **REASSIGN**: Coordinator tăng nhiệm kỳ Fencing Token (*term*), cập nhật bảng ánh xạ sở hữu phân vùng mới vào nhật ký đồng thuận.
 4. **SEEK-RESUME**: Node hồi phục đọc checkpoint Tier-2 mới nhất, nạp state, seek Kafka về *offset + 1* và bắt đầu tiêu thụ dữ liệu.
 5. **COMPLETE**: Coordinator đánh dấu hoàn tất và dọn sạch trạng thái failback tạm thời.
- **Chống trùng lặp & Split-Brain**: Fencing Token (*term, command_id*) đi kèm mọi lệnh điều khiển giúp các Worker từ chối lệnh cũ từ Coordinator cũ bị cô lập mạng (Split-brain). Khi replay luồng Kafka, Worker lọc trùng lặp qua bảng băm định danh sự kiện (có TTL) trong RocksDB cục bộ.